

Računarska grafika
(3OER7O02)

Grafički objekti

Olovke i četke

Vežbe



Grafički objekti

- **Olovke** (*Pen*) – koristi se za iscrtavanje linija i krivih i
- **Četke** (*Brushes*) – za ispunu unutrašnjosti poligona, elipsi i putanja (*paths*),
- **Bitmape** (*Bitmaps*) – za kreiranje, manipulaciju i skladištenje slika na disku, za kopiranje i skrolovanje delova ekrana,
- **Fontovi** (*Fonts*) – za ispis teksta na ekranu i drugim izlaznim uređajima,
- **Logička paleta** (*Logical Palette*) – paleta boja koju aplikacija kreira i dodeljuje je datom DC-ju, definiše skup raspoloživih boja,
- **Putanje** (*Paths*) – jedan ili više oblika koji mogu biti ispunjeni, uokvireni ili i jedno i drugo, i
- **Regioni** (*Regions*) – pravougaonik, poligon ili elipsa (ili kombinacija više oblika), koji se može ispuniti, obojiti, invertovati, uokviriti, koristiti za ispitivanje položaja kursora (*hit-testing*), ili odsecanje (*clipping*).

Atributi grafičkih objekata

Grafički objekat	Atributi
Bitmapa	Veličina [B], dimenzije [pix], color-format, kompresiona šema, itd.
Četka	Stil, boja, šablon i početak
Paleta	Veličina i boje
Font	Ime, širina, visina, debljina, karakter-set, itd.
Putanja	Oblik
Olovka	Stil, debljina i boja
Region	Pozicija i dimenzije

Promena atributa objekata

Kada aplikacija kreira DC, sistem automatski postavlja podrazumevane objekte i njihove attribute (osim bitmape i putanje).

Da bi se promenile podrazumevane vrednosti atributa, mora se:

1. **Napraviti novi objekat odgovarajućeg tipa**
2. **Sačuvati prethodni objekat tog tipa**
3. **Selektovati novi objekat u DC**

Po završetku korišćenja novog objekta, mora se:

1. **Selektovati prethodni objekat u DC**
2. **Obrisati kreirani (novi) objekat**

Primer selektovanja objekta

```
CPen newPen ( PS_SOLID, 0, RGB(0,0,255) );
```

```
CPen* oldPen = pDC->SelectObject( &newPen );
```

```
...
```

```
pDC->SelectObject( oldPen );
```

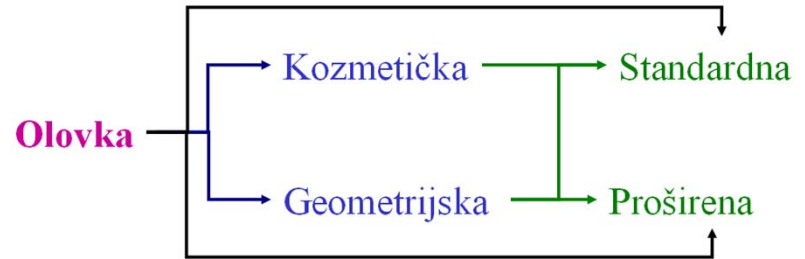
```
newPen.DeleteObject();
```

Olovke

Olovka (*Pen*)

Tipovi olovaka:

- **kozmetička** olovka (*cosmetic*) i
- **geometrijska** olovka (*geometric*)



Kozmetička olovka koristi se kada se zahteva fiksna debljina (1 pixel) i brzo iscrtavanje.

Geometrijska olovka koristi se kada postoji skaliranje linije, definisani spojevi (*joints*), debljina veća od jednog piksela...

Kozmetička olovka

- Dimenzije se definišu u fizičkim jedinicama (*device units*)
- Uvek imaju fiksnu debljinu (1 piksel).
- Linije se iscrtavaju 3-7 puta brže nego sa geometrijskom olovkom.
- Kozmetičke i geometrijske olovke se kreiraju istim naredbama
 - ▷ Atribut debljine se mora postaviti na 0, da bi se kreirala kozmetička olovka.
- Atributi standardne olovke su:
 - ▷ debljina,
 - ▷ stil i
 - ▷ boja.

`CPen::CPen( int nPenStyle, int nWidth, COLORREF crColor );`

Stil kozmetičke olovke

Value	Illustration	Description
PS_SOLID		Puna linija
PS_DASH		Linija sastavljena od crtica
PS_DOT		Linija sastavljena od tačaka (piksela)
PS_DASHDOT		Linija sastavljena od kombinacije crtice pa tačka
PS_DASHDOTDOT		Linija sastavljena od kombinacije crtice pa dve tačke
PS_NULL		Nevidljiva linija
PS_INSIDEFRAME		Puna linija vidljiva samo unutar zatvorenog oblika

Vrednosti parametra *nPenStyle*

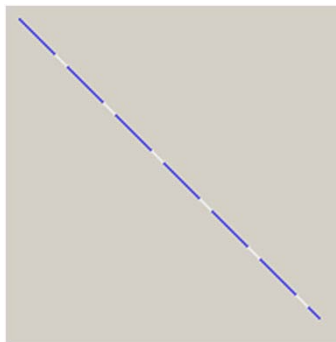
Debljina i boja olovke

- Parametar ***nWidth*** – debljina olovke u logičkim jedinicama
 - Ako je *nWidth* 0, olovka je debljine jednog piksela bez obzira na transformaciju (kozmetička).
 - Debljina se prihvata se jedino ukoliko je stil olovke PS_SOLID.
 - Ako se koristi neki od sledećih stilova: PS_DASH, PS_DOT, PS_DASHDOT, PS_DASHDOTDOT, sistem kreira olovku sa stilom PS_SOLID date debljine.
 - Parametar ***crColor*** – boja olovke
- CPen newPen(PS_DASH, 1, RGB(0,0,255));**

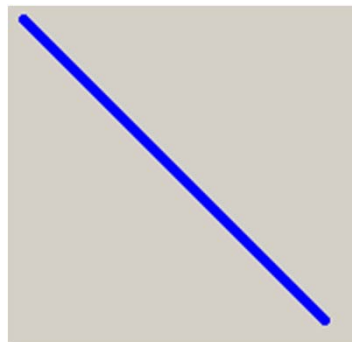
Zanemarivanje stila kod geometrijske olovke

a) `CPen newPen( PS_DASH, 1, RGB(0,0,255) );`

b) `CPen newPen( PS_DASH, 5, RGB(0,0,255) );`



a)



b)

Geometrijska olovka

- Dimenzije se zadaju u logičkim jedinicama.
- Linije crtane ovakvom olovkom mogu se skalirati (biti deblje ili tanje), zavisno od tekuće *world*-transformacije.
- Atributi proširene geometrijske olovke su:
 - ▷ **debljina, stil i boja (kao i kod kozmetičke),**
 - ▷ **šablon (*pattern*),**
 - ▷ **šrafura (*hatch*),**
 - ▷ **stil kraja (*end style*) i**
 - ▷ **stil spoja (*joint style*).**

Indirektno kreiranje standardnih olovaka

Parametri standardne olovke mogu se zadati LOGPEN strukturom

```
typedef struct tagLOGPEN {  
    UINT lopnStyle; // PS_SOLID, PS_DASH ...  
    POINT lopnWidth; // x – debljina, y – ne koristi se  
    COLORREF lopnColor;  
} LOGPEN;
```

A zatim kreirati olovka funkcijom CreatePenIndirect

```
BOOL CPen::CreatePenIndirect( LPLOGPEN lpLogPen );
```

Indirektno kreiranje standardnih olovaka

Parametri proširene olovke definišu se EXTLOGPEN strukturom

```
typedef struct tagEXTLOGPEN {  
    DWORD        elpPenStyle;  
    DWORD        elpWidth;  
    UINT         elpBrushStyle;  
    COLORREF     elpColor;  
    ULONG_PTR    elpHatch;  
    DWORD        elpNumEntries;  
    DWORD        elpStyleEntry[1];  
} EXTLOGPEN;
```

Ali ne postoji funkcija koja kreira olovku na osnovu ove strukture!!!

EXTLOGPEN samo prihvata ono što vraća **GetObject** funkcija.

Kreiranje proširene olovke

Parametri proširene olovke definišu se proširenim konstruktorom

```
CPen::CPen(  
    int nPenStyle, // PS_GEOMETRIC, PS_COSMETIC , ...  
    int nWidth, // pen width  
    const LOGBRUSH *pLogBrush, // pointer to structure for brush attributes  
    int nStyleCount= 0, // length of array containing custom style bits  
    const DWORD* lpStyle // optional array of custom style bits  
);
```

nPenStyle sadrži:

- ▷ tip (PS_GEOMETRIC, PS_COSMETIC)
- ▷ stil (PS_ALTERNATE, PS_SOLID, PS_DASH, ..., **PS_USERSTYLE**, PS_INSIDEFRAME)
- ▷ završetak (PS_ENDCAP_ROUND, PS_ENDCAP_SQUARE, PS_ENDCAP_FLAT)
- ▷ spoj (PS_JOIN_BEVEL, PS_JOIN_MITER, PS_JOIN_ROUND)

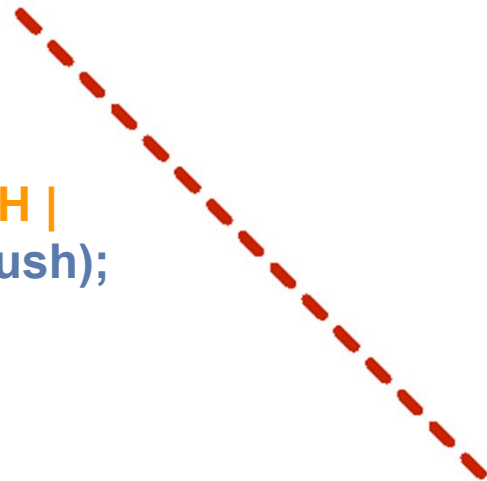
Stilovi proširene olovke

Stil završetka	Stil spoja
PS_ENDCAP_ROUND 	PS_JOIN_BEVEL 
PS_ENDCAP_SQUARE 	PS_JOIN_MITER 
PS_ENDCAP_FLAT 	PS_JOIN_ROUND 

Kreiranje proširene olovke

Primer

```
LOGBRUSH logBrush;  
logBrush.lbStyle = BS_SOLID;  
logBrush.lbColor = RGB(200, 36, 0);  
CPen* pPen = new CPen(PS_GEOMETRIC | PS_DASH |  
PS_ENDCAP_ROUND | PS_JOIN_ROUND, 7, &logBrush);  
CPen* oldPen = pDC->SelectObject(pPen);  
pDC->MoveTo(10, 10);  
pDC->LineTo(300, 300);  
pDC->SelectObject(oldPen);  
delete pPen;
```



Background mod

- Definiše način mešanja pozadinske boje sa drugim bojama bitmapa ili teksta
 - Može biti **OPAQUE** ili **TRANSPARENT**
 - Funkcije za proveru i postavku moda:
 - `int CDC::GetBkMode( );`
 - `int CDC::SetBkMode(int iBkMode);`
- ▷ Obe vraćaju prethodno postavljeni mod



Proba
Proba

Promena boje pozadine i moda

Primer

```
CPen newPen1(PS_DASH, 1, RGB(0, 0, 255));  
CPen newPen5(PS_DASH, 5, RGB(0, 0, 255));
```

```
CPen* pOldPen = pDC->SelectObject(&newPen1);  
int x1 = 20, y1 = 20, x2 = 120, y2 = 120;  
int dx = 80;
```

// Osnovna varijanta

```
pDC->MoveTo(x1, y1);  
pDC->LineTo(x2, y2);
```

// Promena boje pozadine

```
pDC->SetBkColor(RGB(255, 0, 0));  
x1 += dx; x2 += dx;  
pDC->MoveTo(x1, y1);  
pDC->LineTo(x2, y2);
```

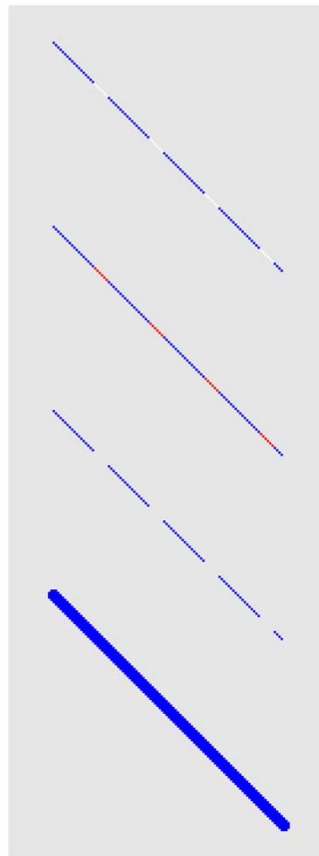
// Promena pozadinskog rezima

```
pDC->SetBkMode(TRANSPARENT);  
x1 += dx; x2 += dx;  
pDC->MoveTo(x1, y1);  
pDC->LineTo(x2, y2);
```

// Promena debljine linije

```
x1 += dx; x2 += dx;  
pDC->SelectObject(&newPen5);  
pDC->MoveTo(x1, y1);  
pDC->LineTo(x2, y2);
```

```
pDC->SelectObject(pOldPen);
```



Drawing mod – 1

Definiše način mešanja *foreground* boje sa drugim bojama olovaka, četki, bitmapa ili teksta, i može biti:

- **R2_BLACK** - Pixel je uvek 0.
- **R2_COPYPEN** - Pixel je boje olovke.
- **R2_MASKNOTPEN** – Piksel je kombinacija boja zajedničkih i za ekran i za inverznu boju olovke.
- **R2_MASKPEN** – Piksel je kombinacija boja zajedničkih i olovci i ekranu.
- **R2_MASKPENN** – Piksel je kombinacija boja zajedničkih i za olovku i za inverznu boju ekrana.
- **R2_MERGEINVERT** – Piksel je kombinacija boje ekrana i inverzne boje olovke.
- **R2_MERGEPALETTE** – Piksel je kombinacija boje olovke i boje ekrana.

Drawing mod – 2

- **R2_MERGEPENNOT** – Piksel je kombinacija boje olovke i inverzne boje ekrana.
- **R2_NOP** – Piksel ostaje nepromenjen.
- **R2_NOT** – Piksel je inverzan u odnosu na boju ekrana.
- **R2_NOTCOPYPEN** - Piksel je inverzan u odnosu na boju olovke.
- **R2_NOTMASKPEN** – Piksel je inverzan u odnosu na R2_MASKPEN boju.
- **R2_NOTMERGEPEN** – Piksel je inverzan u odnosu na R2_MERGEPEN boju.
- **R2_NOTXORPEN** – Piksel je inverzan u odnosu na R2_XORPEN boju.
- **R2_WHITE** – Piksle je uvek 1.
- **R2_XORPEN** – Piksel XOR kombinacija boje olovke i elrana.

Drawing mod – funkcije

Funkcije za očitavanje trenutno selektovanog moda i postavljanje novog moda crtanja:

- `int CDC::GetROP2( );`
- `int CDC::SetROP2(int fnDrawMode);`

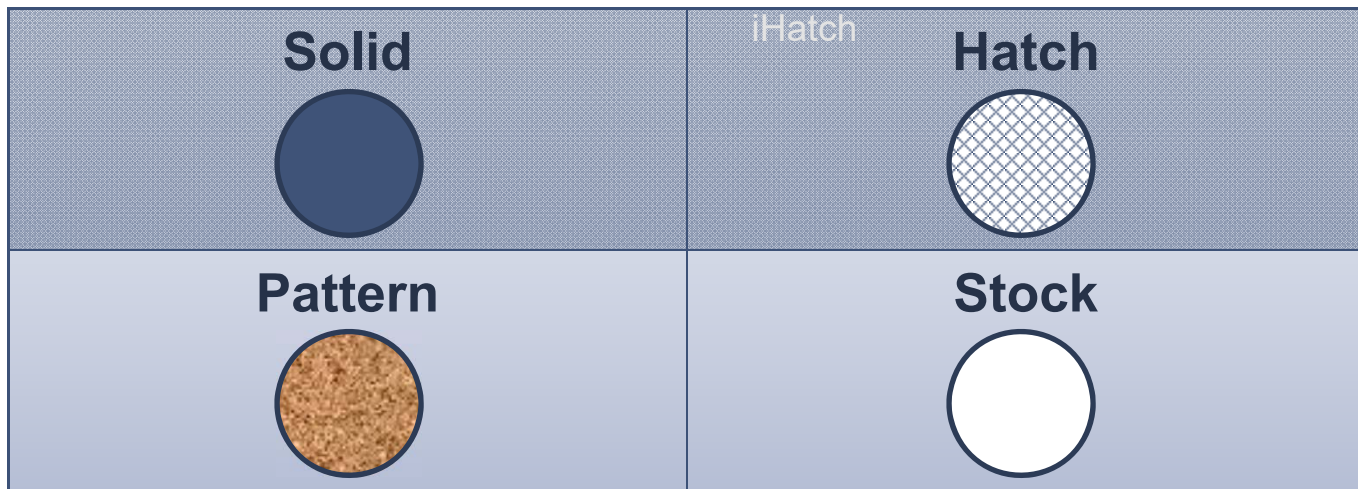


R2_NOTXORPEN

Četke

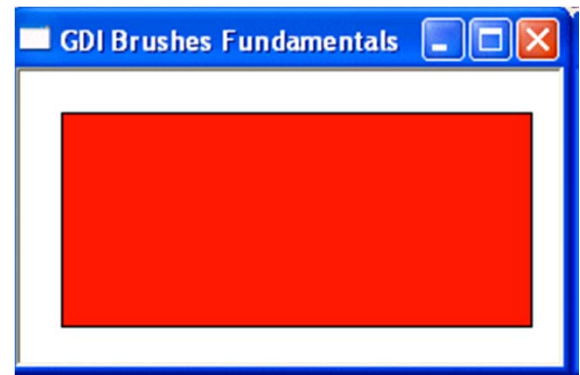
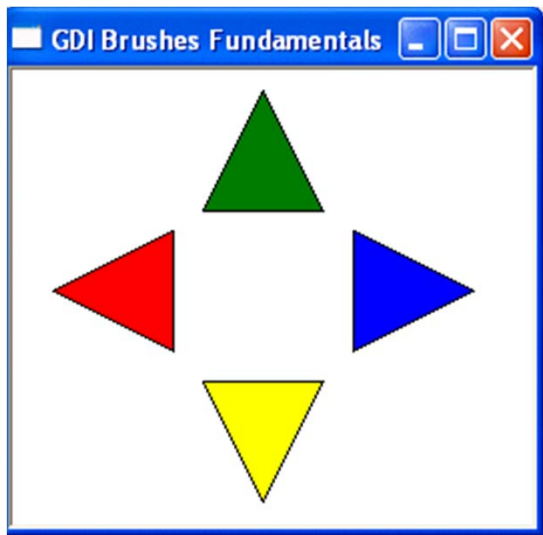
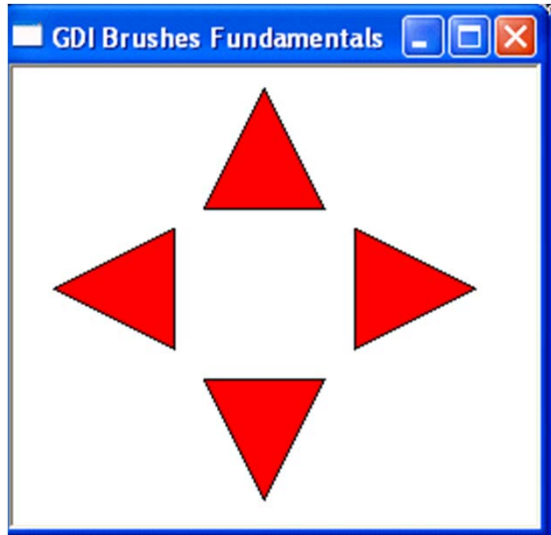
Četka

Četka je grafički objekat koji se koriste za ispunu unutrašnjosti figura kao što su pravougaonik, poligon, elipsa i putanja.



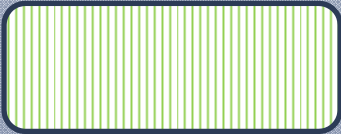
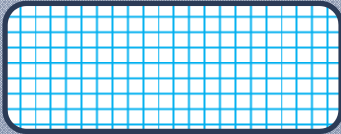
Puna četka (Solid Brush)

`CBrush::CBrush( COLORREF crColor );`



Četka sa šrafurom (Hatched Brush)

`CBrush::CBrush( int nIndex, COLORREF crColor );`

HS_HORIZONTAL 	HS_VERTICAL 	HS_CROSS 
HS_DIAGCROSS 	HS_FDIAGONAL 	HS_BDIAGONAL 

Četka sa bitmapom (Pattern Brush)

■ Ispuna definisana internom bitmapom – DDB (Device Dependent Bitmap)

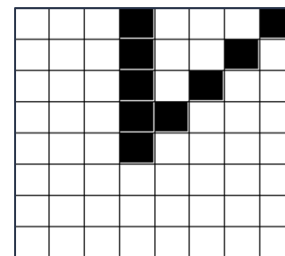
- ▷ `CBrush( CBitmap* pBitmap );`
- ▷ `BOOL CBrush::CreatePatternBrush( CBitmap* pBitmap );`

■ Ispuna definisana eksternom bitmapom – DIB-om (Device Independent Bitmap)

- ▷ `BOOL CBrush::CreateDIBPatternBrush( HGLOBAL hPackedDIB, UINT nUsage );`
- ▷ `BOOL CBrush::CreateDIBPatternBrush( const void* lpPackedDIB, UINT nUsage );`

Četka sa bitmapom (Pattern Brush)

Primer



```
WORD wBits[8] = { 0xee,0xed,0xeb,0xe7,0xef,0xff,0xff,0xff };
```

```
CBitmap brushBmp;
```

```
brushBmp.CreateBitmap(8, 8, 1, 1, wBits);
```

```
CBrush brush;
```

```
brush.CreatePatternBrush(&brushBmp);
```

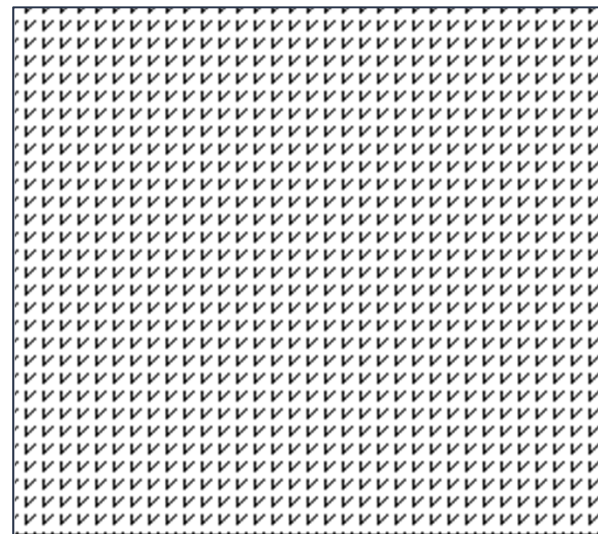
```
CBrush* pOldBrush = pDC->SelectObject(&brush);
```

```
pDC->Rectangle(0, 0, 500, 500);
```

```
pDC->SelectObject(pOldBrush);
```

```
brushBmp.DeleteObject();
```

```
brush.DeleteObject();
```



Indirektno kreiranje četke

Parametri svih vrsta četki mogu se zadati LOGBRUSH strukturom

```
typedef struct tagLOGBRUSH {  
    UINT lbStyle; // BS_SOLID, BS_PATTERN, BS_HATCHED, ...  
    COLORREF lbColor; // DIB_PAL_COLORS, DIB_RGB_COLORS  
    LONG lbHatch; // HS_BDIAGONAL, HS_CROSS, ...  
} LOGBRUSH;
```

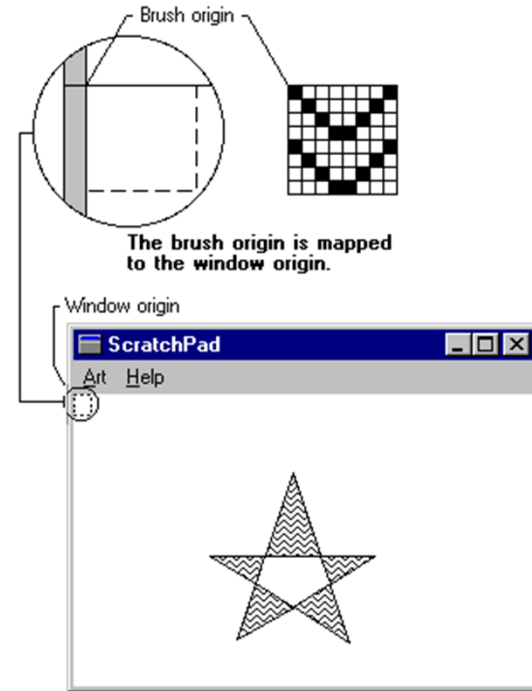
- ▶ Ako je stil četke **BS_PATTERN**, *lbHatch* je hendl bitmape
- ▶ Ako je stil četke **BS_SOLID** ili **BS_HOLLOW**, *lbHatch* se ignoriše.

Četka se kreira metodom CreateBrushIndirect

```
BOOL CBrush::CreateBrushIndirect(const LOGBRUSH*  
    lpLogBrush );
```

Početak četke (origin)

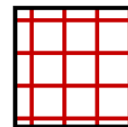
- Piksel (0,0) bitmape koja predstavlja četku se postavlja u koordinatni početak prozora.
 - Ceo prozor se “ispunjava” izabranom bitmapom, pri čemu je ispunjena vidljiva samo u unutrašnjosti figura koje se crtaju.
- CPoint CDC::SetBrushOrg(int x, int y);**
- CPoint CDC::GetBrushOrg();**
- **x** i **y** su koordinate (uređaja) koje predstavljaju novi početak za iscrtavanje bitmape (šrafure) za ispunu
 - Početak četke se pimenjuje na sledeću četku koja će biti selektovana u kontekst uređaja.



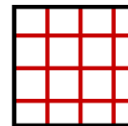
Pomeranje početka četke

Primer

```
CBrush brush(HS_CROSS, RGB(200, 0, 0));  
CBrush* pOldBrush = pDC->SelectObject(&brush);  
pDC->SetBrushOrg(4, 4);  
pDC->Rectangle(CRect(0, 0, 30, 30));  
pDC->SelectObject(pOldBrush);
```



pDC->SetBrushOrg(0, 0);

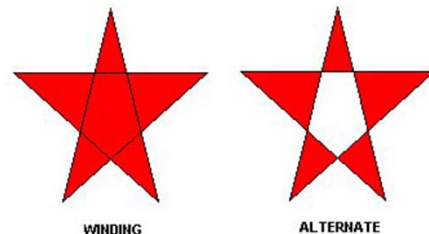


pDC->SetBrushOrg(4, 4);

Polygon-fill mod

Definiše kako se šablon četke koristi za ispunu kompleksnih regiona, i može biti:

- **ALTERNATE** – ispunjava oblasti između neparnog i parnog broja ivica koje preseca scan-linija.
- **WINDING** – ispunjava svaki region sa nenultom *winding* vrednošću.
- Funkcije:
 - `int CDC::GetPolyFillMode( );`
 - `int CDC::SetPolyFillMode(int iPolyFillMode);`



Gotovi GDI objekti

virtual CGdiObject*
CDC::SelectStockObject
(int nIndex)

- BLACK_PEN
- **NULL_PEN**
- WHITE_PEN

- DKGRAY_BRUSH
- GRAY_BRUSH
- **HOLLOW_BRUSH**
- LTGRAY_BRUSH
- **NULL_BRUSH**
- WHITE_BRUSH

GDI+

GDI+ nema koncept selekcije objekata

Objekti se direktno prosleđuju funkcijama za iscrtavanja, npr:

```
Pen pen(Color(255, 0, 0, 0));  
graphics.DrawLine(&pen, 20, 10, 300, 100);
```

Olovka

- Objekat Pen može se kreirati na 2 načina

```
Pen( const Brush* brush, REAL width );
```

```
Pen( const Color& color, REAL width );
```

- Primer

```
Pen pen(Color(255,255,0,0), 4.0f);
```

```
graphics.DrawLine(&pen, 0, 0, 200, 100);
```

Definisanje krajeva linija

```
Pen pen(Color(255, 0, 0, 255), 8);  
stat = pen.SetStartCap(LineCapArrowAnchor);  
stat = pen.SetEndCap(LineCapRoundAnchor);  
stat = graphics.DrawLine(&pen, 20, 175, 300, 175);
```

Četka

■ Puna četka

`SolidBrush( const Color& color );`

■ Četka sa šrafurom

`HatchBrush( HatchStyle hatchStyle, const Color& foreColor, const Color& backColor );`

- Postoji preko 50 predifinisanih tipova šrafura

■ Četka sa bitmapom

`TextureBrush( Image* image, WrapMode wrapMode );`

- `enum WrapMode { WrapModeTile, WrapModeTileFlipX, WrapModeTileFlipY, WrapModeTileFlipXY, WrapModeClamp };`

Četka Primer

```
Color black(255,0,0,0);  
Color white(255,255,255,255);  
HatchBrush brush(HatchStyleHorizontal, black, white);  
graphics.FillRectangle(&brush, 20, 20, 100, 50);  
HatchBrush brush1(HatchStyleVertical, black, white);  
graphics.FillRectangle(&brush1, 120, 20, 100, 50);  
HatchBrush brush2(HatchStyleForwardDiagonal, black, white);  
graphics.FillRectangle(&brush2, 220, 20, 100, 50);  
HatchBrush brush3(HatchStyleBackwardDiagonal, black, white);  
graphics.FillRectangle(&brush3, 320, 20, 100, 50);
```

QPainter

Selektovanje objekata

setPen

setBrush

setFont

setClipPath

setClipRect

setClipRegion

setBackground

setBackgroundMode

setBrushOrigin

setCompositionMode

setLayoutDirection

setOpacity

setRenderHint

setRenderHints

setTransform

setViewTransformEnabled

setViewport

setWindow

setWorldMatrixEnabled

setWorldTransform

Promena atributa objekta

- Moguća direktna promenu atributa selektovanih objekata:

```
QPainter p(this);
```

```
p.setPen(QColor(255,0,0,255));
```

- Moguće je kreiranje novih objekata, a zatim njihova selekcija:

```
QPainter p(this);
```

```
QPen pen;
```

```
pen.setColor(QColor(255, 255, 255, 128));
```

```
p.setPen(pen);
```

Olovka

Kreiranje

- Olovka debljine 1 i proizvoljnog stila

`QPen::QPen(Qt::PenStyle style)`

- Olovka debljine 1 i prosleđene boje

`QPen::QPen(const QColor & color)`

- Olovka sa svim parametrima

`QPen::QPen(const QBrush & brush, qreal width, Qt::PenStyle style = Qt::SolidLine, Qt::PenCapStyle cap = Qt::SquareCap, Qt::PenJoinStyle join = Qt::BevelJoin)`

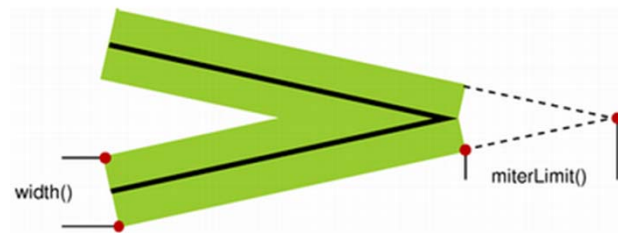
- Kopija druge olovke

`QPen::QPen(const QPen & pen)`

Olovka

Promena atributa

```
void QPen::setBrush(const QBrush & brush)
void QPen::setCapStyle(Qt::PenCapStyle style)
void QPen::setColor(const QColor & color)
void QPen::setCosmetic(bool cosmetic)
void QPen::setDashOffset(qreal offset)
void QPen::setDashPattern(const QVector<qreal> & pattern)
void QPen::setJoinStyle(Qt::PenJoinStyle style)
void QPen::setMiterLimit(qreal limit)
void QPen::setStyle(Qt::PenStyle style)
void QPen::setWidth(int width)
void QPen::setWidthF(qreal width)
```



Četka

- Promena boje četke

`void QBrush::setColor(const QColor & color)`

- Transformacija položaja četke (superponira na transformaciju QPainter-a)

`void QBrush::setMatrix(const QMatrix & matrix)`

`void QBrush::setTransform(const QTransform &matrix)`

- Promena šrafure četke

`void QBrush::setStyle(Qt::BrushStyle style)`

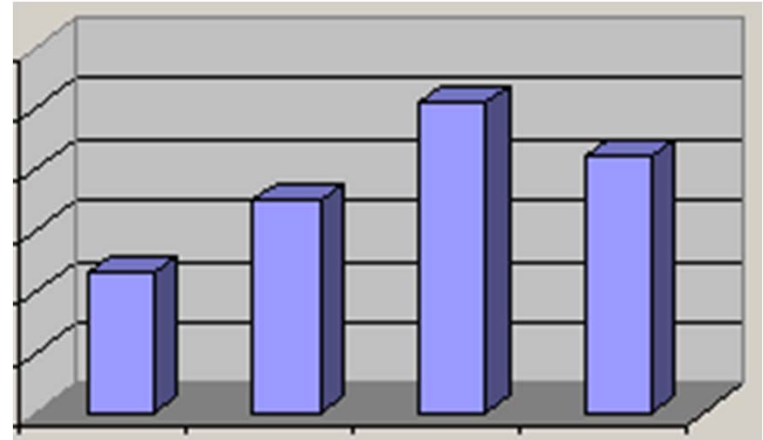
- Promena bitmape četke

`void QBrush::setTexture(const QPixmap & pixmap)`

`void QBrush::setTextureImage(const QImage &image)`

Zadatak za vežbu

Napisati funkciju koja iscrtava 3D dijagram sa stupcima (kao u Excel-u). Ulazni parametri f-je su: okvirni pravougaonik koji definiše gde u prozoru treba iscrtati grafikon (u logičkim koordinatama), vektor realnih brojeva kojim se zadaju vrednosti koje treba predstaviti na grafikonu, dužina prethodnog vektora i boja stubaca (njihove prednje strane). Smatrati da su stupci razmaknuti za 1.5 debljina jednog stupca, i da se njihove visine i debljine menjaju zavisno od zadatog okvirnog pravougaonika.



Pitanja

